Computing Among Four Roots

A Monodromy Model of Reversible Computation

Xiaoyang Liu
University of Waterloo
x98liu@uwaterloo.ca

November 2026

Abstract

How can carrying a root continuously give a complete reversible computation model? We answer this question for the equation $(z^2 - u)^2 = \theta$ over $|u| = 2$ and $|\theta| = 1$. Its four roots at a common control point form the memory. Continuous feedback, read from the carried root, generates returning control and writing motions. Two conditional motions and an unconditional flip realize every permutation of the four roots. Each finite program produces an actual base path whose lifted endpoint agrees with exact symbolic execution. Integer winding numbers give a computable half-turn executor and an inverse that retraces the motion. The construction and its refinement from continuous paths to finite execution are formalized in Lean. The result is a four-state reversible model with one geometric account of state, control, writing, composition, and inversion.

1 A root as a persistent state

A root of an equation is determined locally by its control variables and its current branch. Along a continuous control path, that branch can be carried forward. When the controls return, the root can occupy a different branch. This suggests a concrete computational question: can roots provide the states, while their continuous continuation provides a complete instruction and program semantics?

We give such a model with four states. Classical covering-space theory supplies the lifting principle [1]. Reversible conditional flips and their composition belong to the established theory of reversible computing [2]. The contribution here is their explicit realization in a continuous root space, together with a Lean refinement connecting finite programs, generated paths, and exact execution.

Let $\mathbb{S}^1 = \{q \in \mathbb{C} : |q| = 1\}$. Choose the base

$$B = \{(u, \theta) \in \mathbb{C}^2 : |u| = 2,\ |\theta| = 1\}$$

and take the whole solution space

$$E = \{(u, \theta, z) : (u, \theta) \in B,\ (z^2 - u)^2 = \theta\}. \tag{1}$$

The projection $p : E \to B$ forgets the root. The fiber over the common control point $(2, 1)$ is the memory:

$$M = p^{-1}(2, 1) \simeq \{\sqrt{3}, -\sqrt{3}, 1, -1\}.$$

A state always includes its carried root. Returning the control point to $(2, 1)$ makes the result available as the input to the next instruction.

Proposition 1. *The projection p is a four-sheeted covering. A global coordinate system $H : \mathbb{S}^1 \times \mathbb{S}^1 \to E$ is*

$$H(a, b) = \left(2a^2, b^2, \sqrt{2}\, a\sqrt{1 + \frac{b}{2a^2}}\right), \tag{2}$$

where the square root is principal.

Proof. The quantity $b/(2a^2)$ has norm $1/2$. The argument of the square root therefore has real part at least $1/2$, so the displayed root is continuous. Its square is $2a^2 + b$, which verifies the equation defining E.

For a given point $(u, \theta, z) \in E$, put $b = z^2 - u$. Then $b^2 = \theta$ and $|b| = 1$. Choose $a \in \mathbb{S}^1$ with $2a^2 = u$. The root in (2) has square z^2; choosing the sign of a makes that root equal to z. This proves surjectivity. Moreover, b is recovered from u, z, and

$$a = \frac{z}{\sqrt{2}\sqrt{1 + b/u}} \tag{3}$$

recovers a, proving injectivity. The continuous bijection H is a homeomorphism because its domain is compact and E is Hausdorff. In these coordinates p is the product of two squaring maps, $(a, b) \mapsto (2a^2, b^2)$. Each circle squaring map is a two-sheeted covering. □

For a path $\gamma : x \to y$ and a state in $p^{-1}(x)$, covering-space lifting gives a unique continuous path above γ with that initial state [1, §1.3]. Write $\mathcal{T}_\gamma$ for its endpoint map. With $\gamma * \delta$ meaning that γ is followed by δ,

$$\mathcal{T}_{\gamma * \delta} = \mathcal{T}_\delta \circ \mathcal{T}_\gamma, \qquad \mathcal{T}_{\bar{\gamma}} = \mathcal{T}_\gamma^{-1}. \tag{4}$$

Endpoint-fixed homotopies preserve this transport. The retained branch is thus a memory of the path's action.

2 One continuation rule

The two phases in (2) are observations of the current state: $b = z^2 - u$, and a is given by (3). They provide the feedback quantities used below.

Every state is regular. Indeed, $|b| = 1$, and $z = 0$ would imply $|u| = |b|$, contradicting $|u| = 2$. Differentiating the defining equation therefore gives a single continuation rule,

$$\dot{z} = \frac{2(z^2 - u)\dot{u} + \dot{\theta}}{4z(z^2 - u)}. \tag{5}$$

It applies to control motion, parameter motion, and their combinations.

The formalization establishes both directions of this description. For a real-time curve in E, continuity of the carried state and differentiability of u, θ force z to be differentiable and to satisfy (5). The proof uses the two square relations $b^2 = \theta$ and $z^2 = u + b$. Conversely, take continuous real-time functions u, θ, z satisfying (5) on $[0, 1]$, with the stated derivatives, a nonzero denominator, and a valid initial root. The derivative of $(z^2 - u)^2 - \theta$ is zero throughout the interval, so the root constraint is retained. When the controls lie in B, lift uniqueness identifies the endpoint with $\mathcal{T}_\gamma$. The following feedback motions supply explicit trajectories for this rule.

3 State-dependent returning motions

To let one phase select the motion of the other, choose the continuous weight

$$r(q) = \frac{1 + \operatorname{Re} q}{2} = \frac{|1+q|^2}{4}, \qquad q \in \mathbb{S}^1.$$

It takes values in $[0,1]$, with $r(1) = 1$ and $r(-1) = 0$. Define

$$C_t(a,b) = \left(a e^{\mathrm{i}t\, r(b)/2}, b\right), \tag{6}$$

$$W_t(a,b) = \left(a, b e^{\mathrm{i}t\, r(a)/2}\right). \tag{7}$$

Each is a continuous flow for all real time: its value at zero is the identity, and motion for times s and t composes to motion for $s+t$. The selecting phase remains constant during its instruction, which proves the flow law directly. Through H, these are motions of actual roots in E. Their control velocities are, respectively,

$$(\dot{u}, \dot{\theta}) = (\mathrm{i}u\, r(b), 0), \qquad (\dot{u}, \dot{\theta}) = (0, \mathrm{i}\theta\, r(a)),$$

and the root follows (5). The weight and these feedback laws are specified data of the model.

On M, both phases are signs. At time 2π, C_t flips a exactly when $b = +1$, and W_t flips b exactly when $a = +1$. In every case the base returns to $(2,1)$. We use $C = C_{2\pi}|_M$ as conditional control and $W = W_{2\pi}|_M$ as conditional writing. Retain also the unconditional flip A, obtained at time 2π from

$$A_t(a,b) = \left(a e^{\mathrm{i}t \operatorname{Re} b/2}, b\right).$$

Thus all three instructions are returning continuous feedback motions.

Name the four sign states $p = (+,+)$, $q = (-,+)$, $r = (+,-)$, and $s = (-,-)$. Their actions are

$$C = (p\ q), \qquad W = (p\ r), \qquad A = (p\ q)(r\ s). \tag{8}$$

These are standard reversible conditional flips at the level of their bit actions [2]; here each action has a specified root trajectory.

Proposition 2. *The conditional instructions C, W generate the symmetric group on $\{p,q,r\}$ and fix s. The three instructions A, C, W generate S_4 on M.*

Proof. The transpositions $(p\ q)$, $(p\ r)$ and their conjugate $(q\ r)$ generate the six permutations of $\{p,q,r\}$. For the second assertion, using right-to-left composition of permutations, put $\sigma = CW(AC) = (p\ r\ s\ q)$. The transposition $W = (p\ \sigma(p))$ is adjacent in this four-cycle. Conjugating it by powers of σ gives the neighboring transpositions around the cycle; their compositions give every transposition and hence every permutation of M. □

Uniform open-loop monodromy assigns one base path to the whole fiber. The instruction interface here assigns a path using the current root. Proposition 2 concerns this state-dependent feedback interface on the same covering $p : E \to B$.

4 Finite programs and their paths

Encode the signs by a bijection $\beta : M \to \{0,1\}^2$, with 0 representing $+1$ and 1 representing -1. A program is a finite word in A, C, W, executed from left to right. Its symbolic operations flip the first bit unconditionally for A, the first bit when the second is 0 for C, and the second bit when the first is 0 for W. Applying them in sequence defines $\operatorname{run}(P, x)$.

For an instruction I and current state m, project its root motion over time $[0, 2\pi]$ to obtain a closed base path $\gamma_{I,m}$. For a whole program, concatenate these paths while carrying forward each actual intermediate state. This defines $\gamma_{P,m}$ recursively. The dependence on m is part of the program semantics.

Theorem 3 (Program refinement)**.** *For every finite program P and every initial root $m \in M$,*

$$\beta\big(\mathcal{T}_{\gamma_{P,m}}(m)\big) = \mathrm{run}(P, \beta(m)). \tag{9}$$

Reversing the instruction word restores the initial memory state. Reversing both the word and each motion retraces the lifted computation.

Proof. Each instruction's root trajectory is a continuous lift of its projected path, with the required initial state. Lift uniqueness identifies its endpoint with the geometric instruction. The action in (8) agrees with the corresponding symbolic operation. Equation (4) then proves (9) by induction on the word. Each instruction is an involution on M, so the reversed word restores the initial state. The inverse flow traces each segment backward; applying these inverse motions in reversed order returns along the original segments. □

A complete program is the exchange word CWC. Its bit action is $(x, y) \mapsto (y, x)$, the familiar three-conditional-flip exchange of reversible circuit computation [2]. Starting at the root 1, whose phases are $(+1, -1)$, its actual root states are

$$1 \xrightarrow{C} 1 \xrightarrow{W} \sqrt{3} \xrightarrow{C} -\sqrt{3}. \tag{10}$$

The inverse path follows these segments backward, returning to 1. Thus the example specifies an initial root, an executable word, every instruction endpoint, a generated continuous path, and its inverse.

5 Exact execution by half-turns

For a returning instruction, the current signs $a, b \in \{+1, -1\}$ select an integer winding pair (k, ℓ):

I	k	ℓ
A	b	0
C	$(1+b)/2$	0
W	0	$(1+a)/2$

For every real normalized time t, define the base motion and its lifted phases by

$$(u(t), \theta(t)) = \big(2e^{2\pi \mathrm{i} k t}, e^{2\pi \mathrm{i} \ell t}\big), \tag{11}$$

$$(a(t), b(t)) = \big(a e^{\pi \mathrm{i} k t}, b e^{\pi \mathrm{i} \ell t}\big). \tag{12}$$

Substitution into H gives a continuous root path over (11) with the selected initial root. These formulas equal the corresponding feedback motion at physical time $2\pi t$. At $t = 1$,

$$(a, b) \longmapsto \big(a(-1)^k, b(-1)^\ell\big). \tag{13}$$

This half-turn rule is an exact executor: it follows integer windings and retains their effect on the carried phases. Each endpoint root has the exact name 1, -1, $\sqrt{3}$, or $-\sqrt{3}$.

For inverse execution, reverse the word and negate each selected winding pair. Formula (12) at negative time supplies the inverse motion. The selecting coordinate is conserved during each instruction, so its winding selection agrees when that instruction is traversed backward. The executor therefore reconstructs the inverse path as well as the inverse endpoint.

The command line exposes this execution directly:

```
lake exe monodromy 1 CWC
lake exe monodromy -inverse -sqrt3 CWC
lake exe monodromy -trace 1 CWC
```

The first two commands realize the forward and inverse computations in (10). The trace option reports the selected windings. The command syntax accepts the root names `1`, `-1`, `sqrt3`, and `-sqrt3`, and words over `A`, `C`, `W`. Running `lake exe monodromy` gives the round trip $1 \to -\sqrt{3} \to 1$.

6 Formalization and verification

Lean is the native environment for the construction and its proofs [3]. The development reuses mathlib's analysis, covering spaces, permutation groups, and continuous flows [4]. The source follows the mathematical dependencies. `Algebra.lean` and `Covering.lean` supply the common action and lifting results. `Quartic.lean` constructs E and its phase observations; `Paths.lean` connects carried-root paths with differentiable continuation; `Flows.lean` defines the returning feedback instructions; `Programs.lean` gives finite-word semantics and Theorem 3. The computable definitions are collected in `Execution.lean`, and `Lifting.lean` connects their integer half-turn execution to the actual covering-space transport.

In particular, `path_correct` and `bits_runGeometry` connect program paths, root states, and symbolic readout. `follow_transport` proves the exact half-turn rule for arbitrary integer winding pairs. The all-time phase identity and the inverse-segment identity connect the executor's forward and inverse paths to the specified continuous feedback motions. Initialization and readout use β^{-1} and β throughout.

The accompanying source is fixed to Lean 4.35.0 and the mathlib revision

```
56a005855ea17e0b0958f73ec801ae140b0a1679
```

and can be checked from the project root with

```
lake build
lake env lean src/Audit.lean
```

The axiom audit permits `propext`, `Quot.sound`, and `Classical.choice`. Proofs are checked through Lean's kernel with these standard dependencies. The verification scope comprises the formal mathematical definitions, geometric refinement, and pure executor. Parsing, rendering, compilation, and the execution platform supply the surrounding command-line environment.

The model has precisely four memory states, specified feedback laws, and finite instruction words. Its expressive result is the complete permutation group of those states. Its continuous realization and exact execution share the same lifting semantics: a root persists while control and writing paths move it, and the resulting root becomes the next state.

AI contributions

Generative AI (OpenAI ChatGPT and Codex, including collaborating AI agents) contributed substantially to the conception and mathematical development of this work, including research ideation, theoretical constructions, conjectures, proofs, Lean formalization, implementation, and manuscript preparation.

The human author directed the research, determined its scope and principles, established its aesthetic aims and the use of Lean as the native research environment, critically evaluated the results, and assumes full responsibility for the work.

The formalization and source code accompany this manuscript for independent verification.

Openness

The manuscript and the project's original code are released under The Unlicense. *No rights reserved.* Cited works and third-party dependencies retain their respective terms.

References

[1] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2002, §1.3. https://pi.math.cornell.edu/~hatcher/AT/ATpage.html.

[2] Tommaso Toffoli. *Reversible Computing*. Technical Memorandum MIT/LCS/TM-151, Massachusetts Institute of Technology, February 1980. https://hdl.handle.net/1721.1/148978.

[3] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. In *Automated Deduction—CADE 28*, LNCS 12699, pp. 625–635, Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[4] The mathlib Community. *The Lean Mathematical Library*. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pp. 367–381, ACM, 2020. https://doi.org/10.1145/3372885.3373824.